

Git

DevOps Git Cheat Sheet

Everyday command reference — 90+ commands across 12 categories

destructive — Permanently removes data

rewrites history — Use with care on shared branches

SETUP & CONFIG

<code>git config --global user.name "Name"</code>	Set your global display name for all commits.
<code>git config --global user.email "email"</code>	Set your global email address for commits.
<code>git config --global core.editor vim</code>	Set the default text editor for commit messages.
<code>git config --list</code>	Display all active config settings.
<code>git config --global alias.st status</code>	Create a shortcut alias — 'st' runs 'status'.

STASH

<code>git stash</code>	Shelve all tracked changes for a clean directory.
<code>git stash push -m "label"</code>	Stash with a descriptive label.
<code>git stash list</code>	List all stashed changesets.
<code>git stash pop</code>	Apply the most recent stash and remove it.
<code>git stash apply stash@{n}</code>	Apply a specific stash without removing it.
<code>git stash drop stash@{n}</code>	Delete a specific stash entry.
<code>git stash branch</code>	Create a branch from a stash.

INIT & CLONE

<code>git init</code>	Initialize a new empty repository in current directory.
<code>git init</code>	Create a new repo inside a named directory.
<code>git clone</code>	Download a full copy of a remote repository locally.
<code>git clone</code>	Clone into a custom local directory name.
<code>git clone --depth 1</code>	Shallow clone — fetch only the latest snapshot.

TAGS & RELEASES

<code>git tag</code>	List all existing tags.
<code>git tag v1.0.0</code>	Create a lightweight tag at the current commit.
<code>git tag -a v1.0.0 -m "msg"</code>	Create an annotated tag — preferred for releases.
<code>git push origin --tags</code>	Push all local tags to the remote.
<code>git push origin v1.0.0</code>	Push a single specific tag to the remote.
<code>git tag -d v1.0.0</code>	Delete a tag locally.
<code>git push origin --delete v1.0.0</code>	Delete a tag on the remote repository.

STAGING & COMMITS

UNDO & RECOVERY

<code>git status</code>		Show modified, staged, or untracked files.	<code>git restore</code>	destructive	Discard unstaged changes — reverts to last commit.
<code>git add</code>		Stage a specific file for the next commit.	<code>git restore --staged</code>		Unstage a file while keeping working directory changes.
<code>git add .</code>		Stage all modified and new files.	<code>git revert</code>		Create a new commit that undoes a previous one.
<code>git add -p</code>		Interactively stage individual hunks.	<code>git reset --soft HEAD~1</code>		Undo last commit but keep changes staged.
<code>git commit -m "msg"</code>		Record staged changes with an inline message.	<code>git reset --mixed HEAD~1</code>		Undo last commit and unstage changes.
<code>git commit --amend</code>	rewrites history	Edit the last commit message or add forgotten files.	<code>git reset --hard HEAD~1</code>	destructive	Undo last commit AND discard all changes permanently.
<code>git commit --amend --no-edit</code>	rewrites history	Amend last commit without changing its message.	<code>git reflog</code>		View every HEAD movement — lifeline to recover commits.
			<code>git cherry-pick</code>		Apply changes from a specific commit onto current branch.

BRANCHING

<code>git branch</code>		List all local branches; active one marked with *.	<code>git grep "pattern"</code>		Search for a string across all tracked files.
<code>git branch -a</code>		List all branches including remote-tracking refs.	<code>git bisect start</code>		Start a binary search to find when a bug was introduced.
<code>git branch</code>		Create a new branch at current HEAD.	<code>git bisect good</code>		Mark a commit as bug-free during a bisect session.
<code>git switch</code>		Switch to an existing branch.	<code>git bisect bad</code>		Mark a commit as broken during a bisect session.
<code>git switch -c</code>		Create and immediately switch to a new branch.	<code>git show</code>		Display metadata and diff of a specific commit.
<code>git branch -d</code>		Delete a fully merged local branch.	<code>git ls-files</code>		List all files tracked by Git in the current repo.
<code>git branch -D</code>	rewrites history	Force-delete a branch regardless of merge status.			
<code>git branch -m</code>		Rename a local branch.			

MERGE & REBASE

SEARCH & INSPECT

SUBMODULES & WORKTREES

<code>git merge</code>		Merge a branch into current (creates merge commit).	<code>git submodule add</code>	Embed another repo as a submodule inside the project.
<code>git merge --no-ff</code>		Always create a merge commit, preserving history.	<code>git submodule update --init --recursive</code>	Initialize and update all submodules after a clone.
<code>git merge --squash</code>		Squash all branch commits into one staged change.	<code>git submodule foreach git pull</code>	Run git pull in every submodule at once.
<code>git rebase</code>	rewrites history	Re-apply commits on top of another branch.	<code>git worktree add</code>	Check out a branch into a separate directory.
<code>git rebase -i HEAD~N</code>	rewrites history	Interactive rebase to squash, edit or reorder commits.	<code>git worktree list</code>	List all active worktrees linked to this repo.
<code>git rebase --abort</code>		Cancel an in-progress rebase and restore state.	<code>git worktree remove</code>	Remove a linked worktree when done with it.
<code>git rebase --continue</code>		Resume a paused rebase after resolving conflicts.		

REMOTE OPERATIONS

<code>git remote -v</code>		List all configured remotes with their URLs.
<code>git remote add origin</code>		Connect a local repo to a remote named 'origin'.
<code>git remote set-url origin</code>		Change the URL of the 'origin' remote.
<code>git fetch</code>		Download remote changes without merging.
<code>git fetch --prune</code>		Fetch and delete refs to remotely-deleted branches.
<code>git pull</code>		Fetch and merge remote changes into current branch.
<code>git pull --rebase</code>		Fetch and rebase — keeps history linear.
<code>git push origin</code>		Push a local branch to the remote.
<code>git push -u origin</code>		Push and set upstream tracking for future pushes.
<code>git push --force-with-lease</code>	rewrites history	Force-push only if no one else pushed since last fetch.
<code>git push origin --delete</code>		Delete a branch on the remote repository.

LOG & HISTORY

<code>git log</code>	Show the full commit history.
<code>git log --oneline --graph --all</code>	Compact graphical view of all branches.
<code>git log -p</code>	Show every commit that touched a file with its diff.
<code>git log --author="name"</code>	Filter commit history to a specific author.
<code>git log --since="2 weeks ago"</code>	Show commits within a relative time window.
<code>git shortlog -sn</code>	Summary of commit counts per author.
<code>git blame</code>	Annotate each line with the last commit that changed it.
<code>git diff</code>	Show unstaged changes in the working directory.
<code>git diff --staged</code>	Show changes staged for the next commit.
<code>git diff ..</code>	Compare changes between two branches.